

Arquitectura de Integración

MODERNIZACIÓN DE SISTEMAS BANCARIOS

JOSÉ LUIS RODRÍGUEZ

1. Resumen Ejecutivo

Este documento presenta la propuesta de arquitectura de integración para la modernización de sistemas de un banco tradicional, cubriendo la integración de un Core Bancario tradicional con un nuevo Core Bancario digital, web banking y móvil banking, una plataforma de pagos, APIs de Open Finance, gestión de riesgos y prevención de fraude.

La propuesta sigue un enfoque top-down de Arquitectura Empresarial. A partir de los flujos de valor del negocio se identificaron las capacidades necesarias, se agruparon en dominios utilizando BIAN como referencia y, posteriormente, se definieron los sistemas de registro y la arquitectura de integración.

2. Alcance y Escenario

Para contextualizar el ejercicio, se toma como referencia a Banco Pichincha. El escenario propuesto contempla modernizar la infraestructura tecnológica para mejorar los servicios digitales y atender las necesidades de integración y regulación descritas en el enunciado. Se requiere integrar:

- Core Bancario tradicional más un nuevo Core Bancario digital.
- Nuevo sistema web banking y móvil banking.
- Plataforma de servicios de pago.
- APIs para servicios de terceros (Open Finance).
- Sistema de gestión de riesgos.
- Sistema de prevención de fraudes.

3. Metodología: Enfoque de Arquitectura Empresarial

El diseño parte de los flujos de valor (value streams), considerando la secuencia de actividades que entregan valor a un stakeholder. De ahí se desciende a capacidades de negocio, luego a dominios, y al final a la arquitectura de aplicaciones y tecnología.

- **1. Flujos de valor:** Qué valor se entrega y a quién.
- **2. Capacidades de negocio:** Qué puede hacer el negocio.
- **3. Dominios:** Cómo se agrupan las capacidades (validado con BIAN).
- **4. Aplicaciones y tecnología:** Sistemas, APIs, eventos.

Se eligió este enfoque porque permite justificar la decisión tecnológica a partir de una necesidad de negocio identificada previamente

4. Flujos de Valor

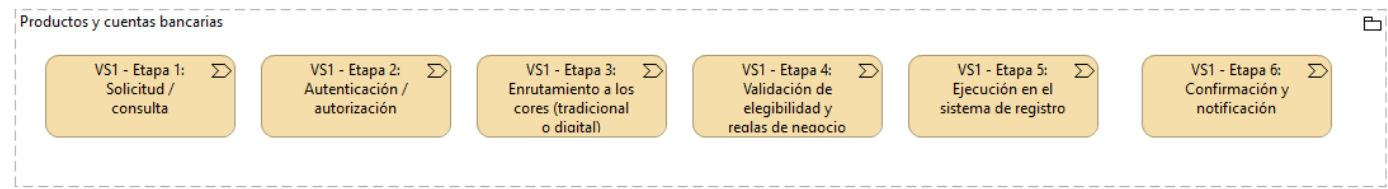
Se identificaron tres flujos principales:

#	Flujo de valor	Disparador	Valor entregado
1	Productos y cuentas bancarias	Cliente (directo o vía tercero autorizado) solicita/consulta un producto	Producto activo o información entregada
2	Banca digital (self-service)	Cliente inicia sesión en web/móvil	Consulta o transacción resuelta sin intervención humana
3	Procesamiento de pagos	Cliente (directo o vía tercero autorizado) origina una orden de pago	Fondos movidos, liquidados y confirmados

4.1 Etapas de los flujos

Ejercicio de Arquitectura de Integración

Flujo 1 — Productos y cuentas bancarias

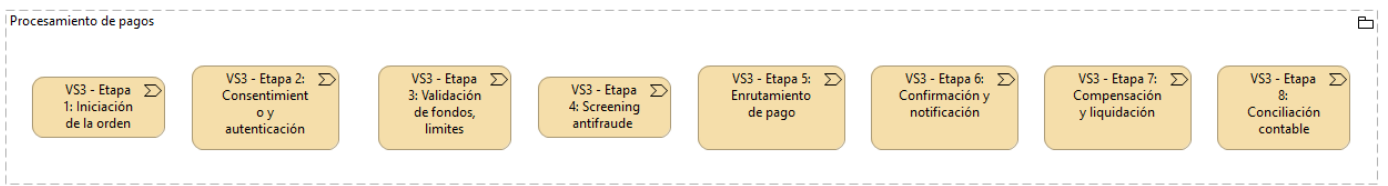


Nota: este flujo no sincroniza los cores entre sí en la operación habitual.

Flujo 2 — Banca digital (self-service)



Flujo 3 — Procesamiento de pagos



En el flujo de pagos, la etapa «Confirmación y notificación» se interpreta como la confirmación de recepción de la orden. La confirmación definitiva del resultado se comunica una vez conocido el resultado de la compensación y liquidación, cuando corresponda al tipo de pago.

5. Mapa de Capacidades de Negocio

Flujo	Capacidad (L1)	Sub-capacidades (L2)	Service Domain(s) BIAN
F1	Originación y administración de productos bancarios	Originar producto	Customer Offer, Product Deployment
		Administrar cuenta/préstamo	Current Account, Savings Account, Consumer/Mortgage Loan
		Verificar identidad y cumplimiento	Party Authentication/Authorization, KYC
F2	Experiencia de banca digital	Gestionar permisos y restricciones de acceso al canal	Customer Access Entitlement
		Autenticar cliente (multi-nivel)	Party Authentication
		Mantener sesión técnica de interacción	Sin Service Domain BIAN
		Registrar y analizar actividad del canal	Channel Activity Analysis, Customer Event History
F3	Procesamiento de pagos	Iniciar orden de pago	Payment Order Initiation
		Validar cumplimiento del pago	Sanction Screening
		Ejecutar y enrutar pago	Payment Execution
		Compensar y liquidar / Conciliar	Clearing and Settlement, Reconciliation
		Confirmar y notificar	Payment Order

Ejercicio de Arquitectura de Integración

Flujo	Capacidad (L1)	Sub-capacidades (L2)	Service Domain(s) BIAN
Compartida	Gestión de riesgo	Evaluar riesgo crediticio / operacional	Credit Risk Rating, Operational Risk
	Prevención de fraude	Detectar / resolver fraude	Fraud Detection, Fraud Resolution
Habilitadora	Observabilidad y trazabilidad	Monitoreo de disponibilidad, logging técnico	IT Service Availability
	Integración e interoperabilidad	Exponer APIs y enrutar solicitudes	Sin equivalencia directa asignada
	Integración e interoperabilidad	Adaptar protocolos y transformar mensajes	Sin equivalencia directa asignada

6. Dominios y Bounded Contexts

Se aplican principios DDD (responsabilidad única, ownership claro de datos, sin bases compartidas) para derivar los dominios a partir de las capacidades validadas.

Dominio	Datos bajo responsabilidad	Clasificación DDD	Interacción principal
Gestión de identidad y de personas y organizaciones	Perfil, credenciales, consentimientos	Generic	Síncrono (API)
Originación y administración de productos bancarios	Cuenta, préstamo, oferta	Core	Síncrono (API) + eventos de estado
Cumplimiento y KYC	Expediente KYC, resultado de verificación	Supporting	Síncrono (API, bloqueante en originación)
Riesgo	Score de riesgo, apetito/límites	Core	Síncrono (camino crítico)
Fraude	Score de fraude, caso de investigación	Core	Síncrono baja latencia + evento
Pagos	Orden de pago, transacción, liquidación	Core	Evento + síncrono puntual
Gestión de interacciones y canales digitales	Historial de interacción, persistido en la Plataforma de Observabilidad (sección 7); la sesión es efímera (JWT), no persistida por este dominio.	Supporting	Síncrono (API) para la operación; evento para registro de historial

7. Matriz de Sistemas de Registro

Dominio	Sistema de Registro	Tipo	Rol
Originación y administración de productos bancarios	Core Bancario Tradicional (legado)	Legacy	Autoritativo para productos no migrados
	Core Bancario Digital (nuevo)	Nuevo / vendor	Autoritativo para productos nuevos y migrados
Cumplimiento y KYC	Plataforma KYC	COTS / utility	Expediente KYC y gestión de derechos del titular
Riesgo	Motor de Riesgo	Nuevo / vendor scoring	Score de riesgo y límites
Fraude	Motor de Detección de Fraude	Nuevo / vendor ML	Score de fraude y casos
Pagos	Plataforma de Pagos (Payment Hub)	Nuevo / vendor payment hub	Orden, liquidación, conciliación
Gestión de identidad y de personas y organizaciones	Plataforma IAM	COTS / SaaS	Identidad, credenciales, consentimientos y perfil

Ejercicio de Arquitectura de Integración

Dominio	Sistema de Registro	Tipo	Rol
Gestión de interacciones y canales digitales	Plataforma de Banca Digital (BFF)	Nuevo específico	Canal de interacción
Gestión de interacciones y canales digitales	Historial de Interacción (repositorio lógico alojado en la Plataforma de Observabilidad)	Lógico, sobre COTS/SaaS	Repositorio persistente y autoritativo del historial de interacción

8. Arquitectura de Alto Nivel — Diagramas C4

8.1 Diagrama de Contexto (C4 — Nivel 1)

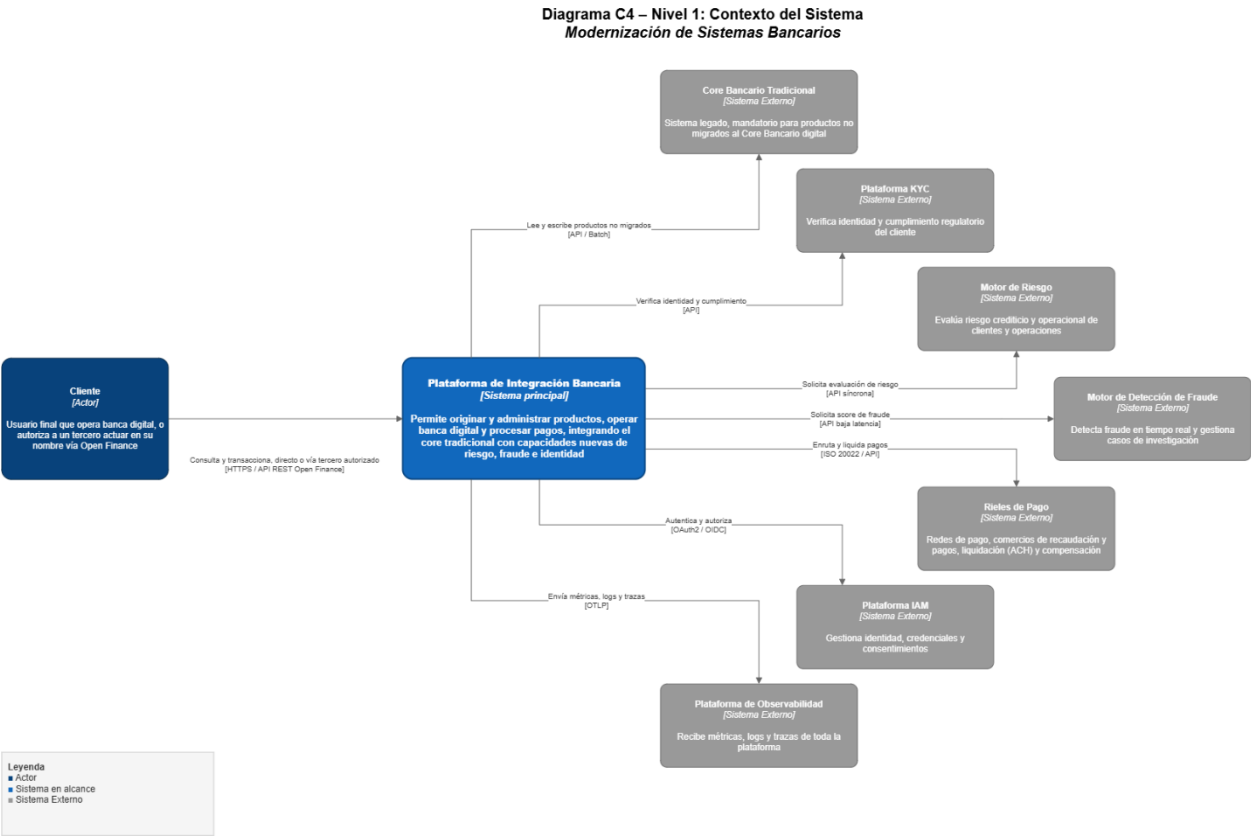


Figura 1. Diagrama C4 de Contexto: Plataforma de Integración Bancaria.

El Core Bancario Tradicional se modela como sistema externo, reflejando que en el patrón Strangler Fig es un sistema existente que se integra.

8.2 Diagrama de Contenedores (C4 — Nivel 2)

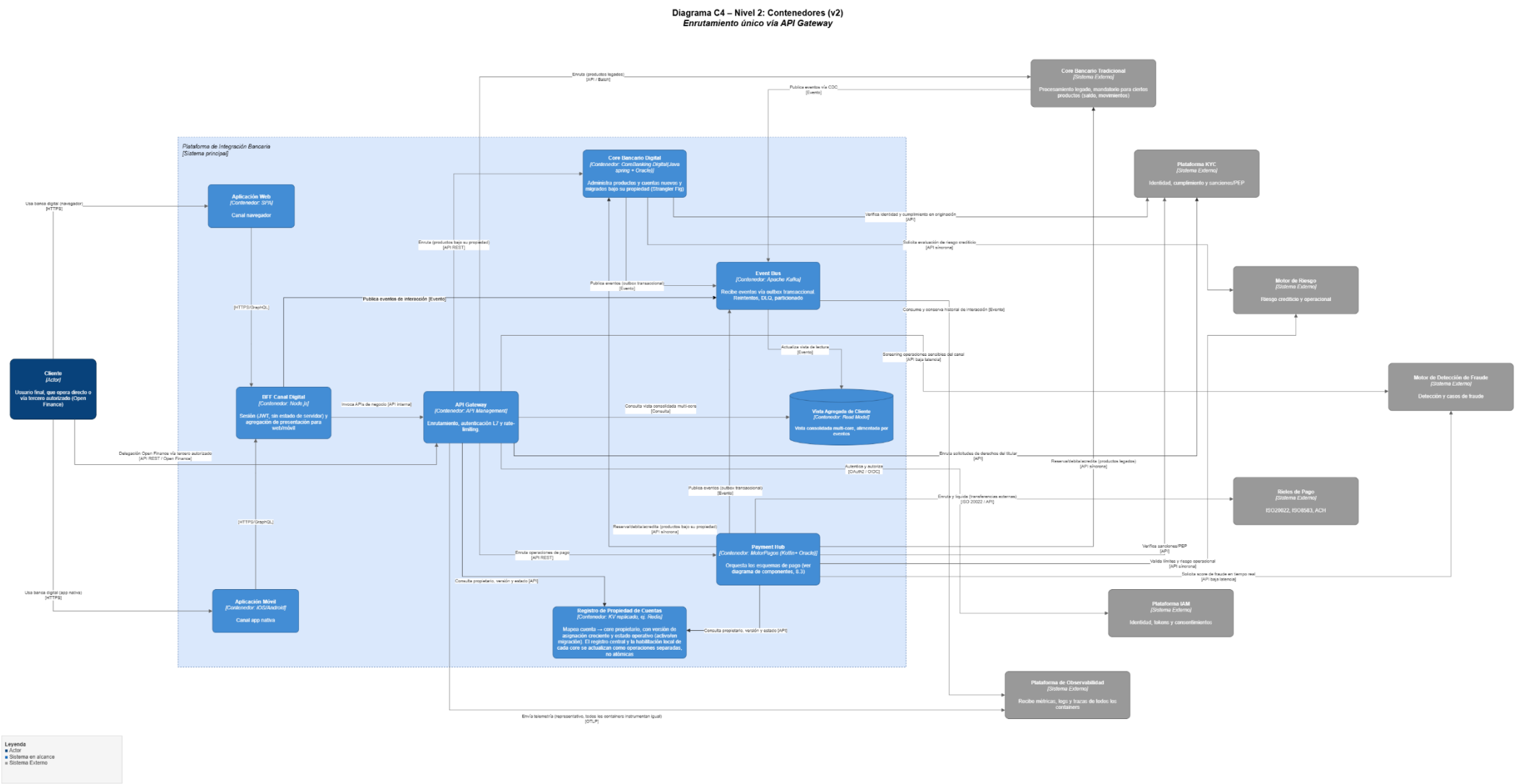


Figura 2. Diagrama C4 de Contenedores: enrutamiento único desde los canales hacia el Gateway; llamadas dominio-a-dominio y dominio-a-externo directas donde corresponde.

Ejercicio de Arquitectura de Integración

Container	Rol
Aplicación Web / Aplicación Móvil	Canales de cliente (SPA y app nativa), dentro del alcance de la plataforma; consumen el BFF
BFF Canal Digital	Sesión (JWT, sin estado de servidor) y agregación de presentación
API Gateway	Enrutamiento técnico, authN/Z L7 y rate-limiting para todo lo que entra desde los canales. No ejecuta lógica de negocio ni orquesta procesos multi-dominio.
Core Bancario Digital	Productos/cuentas nuevos y migrados bajo su propiedad (Strangler Fig)
Payment Hub	Orquesta el esquema de pago (ver diagrama de componentes, 8.3)
Registro de Propiedad de Cuentas	KV replicado (ej. Redis); mapea cuenta → core propietario con número de versión, validado por cada core al aceptar una escritura
Event Bus	Recibe eventos vía outbox transaccional; reintentos, DLQ, particionado
Vista Agregada de Cliente	Read model multi-core, evita bases de datos compartidas

La Plataforma de Observabilidad recibe métricas, logs y trazas de todos los contenedores. La conexión API Gateway → Plataforma de Observabilidad representa el envío de telemetría mediante OTLP; la instrumentación también aplica al BFF, Core Digital, Payment Hub, Event Bus y demás contenedores.

La conexión Event Bus → Plataforma de Observabilidad representa un flujo distinto: la entrega de eventos de interacción publicados por el BFF. La plataforma consume estos eventos y los conserva en el repositorio lógico de Historial de Interacción, cuya propiedad funcional corresponde al dominio de Gestión de interacciones y canales digitales, conforme a los capítulos 6 y 7. Este repositorio mantiene políticas propias de acceso, retención y recuperación. El BFF publica los eventos, pero no persiste el historial.

Alcance de "enrutamiento único": aplica a la entrada de tráfico desde los canales (Web, Móvil, BFF) y a la delegación Open Finance de un tercero autorizado. .

Débito/crédito real: Payment Hub reserva, debita o acredita contra el core que sea propietario de la cuenta (Digital o Tradicional), llamando directamente a la API de ese core. Esta llamada no pasa por el API Gateway porque no es tráfico de ingreso desde un canal.

8.3 Diagrama de Componentes — Payment Hub (C4 — Nivel 3)

Se detalla a nivel de componentes el contenedor Payment Hub, incluyendo sus dependencias externas (otros containers y sistemas externos), ya que es donde vive la orquestación del esquema de pago y donde se resuelve el patrón de idempotencia/outbox exigido por el procesamiento de pagos.

Diagrama C4 – Nivel 3: Componentes de Payment Hub
Incluye dependencias externas del contenedor

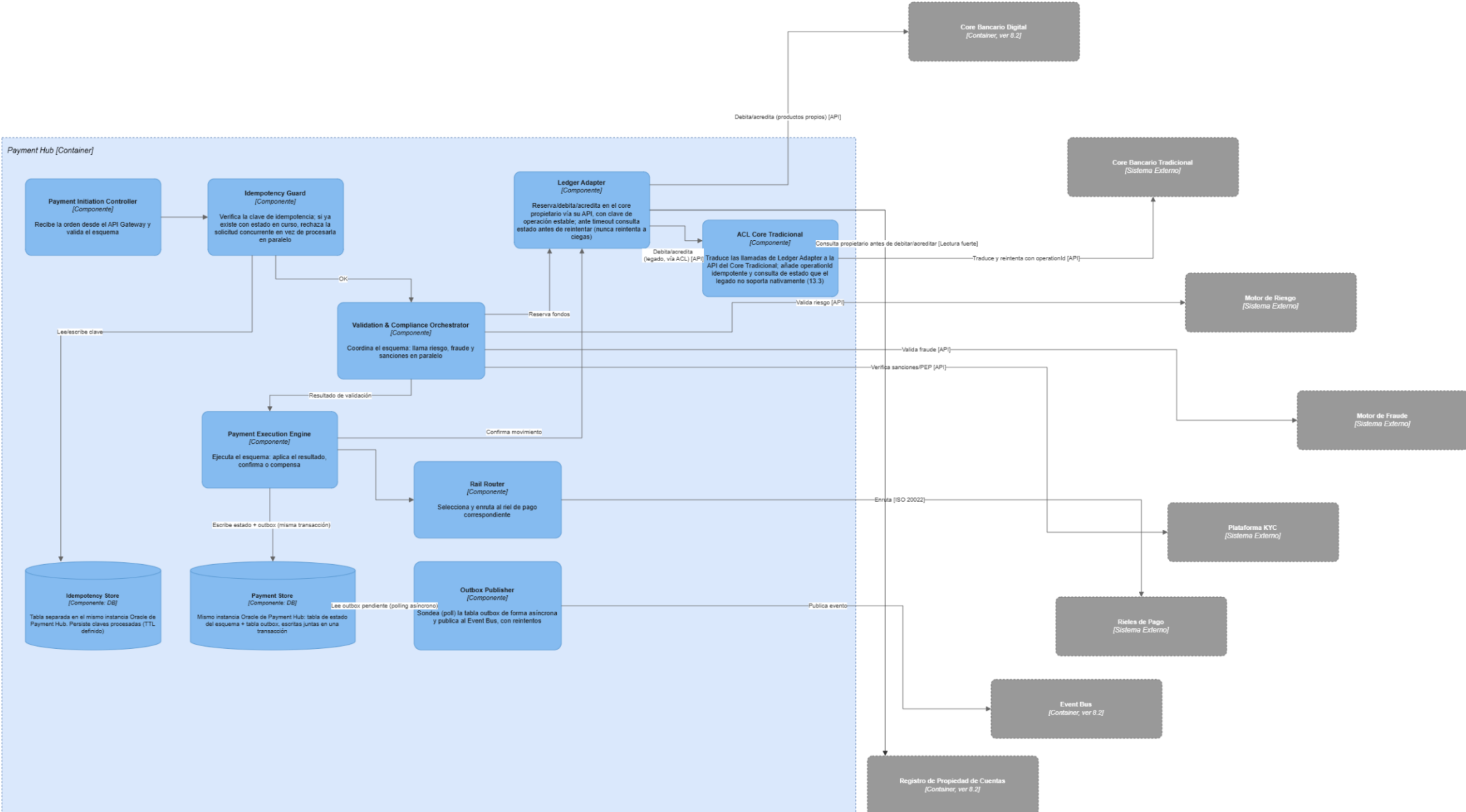


Figura 3. Diagrama C4 de Componentes: Payment Hub, con dependencias externas explícitas.

Ejercicio de Arquitectura de Integración

En este diagrama, la consulta al Registro de Propiedad obtiene el propietario, la versión y el estado. La protección de las escrituras requiere además la validación local del core. El ACL solo puede ofrecer identificación y consulta de operaciones si el legado dispone de una referencia verificable; los reintentos se realizan conforme al tratamiento de resultados inciertos de la sección 13.3.

Cómo resuelve el problema de dual-write: Payment Execution Engine escribe el estado del pago y la fila outbox en la misma transacción de base de datos (ambas viven en el Payment Store, misma instancia Oracle de Payment Hub). Outbox Publisher, de forma asíncrona y separada, sondea (poll) esa tabla outbox y publica al Event Bus con reintentos. Así, si la escritura del estado tiene éxito, el evento eventualmente se publica; y si la escritura falla, no hay evento huérfano. Idempotency Guard, respaldado por su propia tabla en la misma instancia Oracle, evita reprocesar la misma orden ante reintentos del cliente o del Gateway.

El Ledger Adapter, el Validation & Compliance Orchestrator y el Rail Router llaman directamente a sus dependencias externas (cores, motores de riesgo/fraude, KYC, rieles de pago). El diagrama las muestra explícitamente como referencias, ya detalladas en el diagrama de contenedores (8.2) o en el de contexto (8.1).

9. Patrones de Integración y Tecnologías

La arquitectura combina APIs síncronas para operaciones que requieren respuesta inmediata y eventos para propagar cambios y actualizar información derivada. Los patrones se aplican según la responsabilidad de cada componente, manteniendo la propiedad de los datos en los dominios y la coordinación de procesos de negocio en sus servicios responsables.

Patrón / Tecnología	Dónde se aplica	Por qué
Backend for Frontend (BFF)	Atención de los canales web y móvil	Adapta solicitudes y respuestas a las necesidades de presentación. Las llamadas a APIs de negocio pasan por el Gateway. La publicación de eventos de interacción al Event Bus constituye una integración asíncrona separada.
API Gateway (puro enrutamiento)	Único punto de entrada para tráfico de ingreso desde canales y Open Finance	Centraliza validación de identidad y permisos, límites de consumo y enrutamiento según el registro de propiedad. Las reglas y decisiones de negocio se ejecutan en los servicios responsables.
CQRS / Read Model	Vista Agregada de Cliente	Desacopla lectura de escritura. Separa las consultas consolidadas de las operaciones transaccionales. La vista se alimenta por eventos o CDC y admite consistencia eventual; no se utiliza para validar fondos ni autorizar movimientos.
Event-Driven Architecture	Backbone entre dominios (Event Bus)	Desacopla productores/consumidores; permite propagar cambios sin que el productor tenga que esperar a todos los consumidores
Change Data Capture (CDC)	Core Tradicional → Event Bus	Integra el legado sin acoplarlo síncronamente, condicionado a que el legado exponga una fuente de captura de cambios viable (log de transacciones, triggers u otro mecanismo) y a los permisos que su administrador otorgue sobre ella. No se garantiza en general que la integración sea posible "sin modificarlo"; si el legado no ofrece esa capacidad, se requiere evaluar una alternativa (p. ej., extracción periódica) antes de comprometerse con este patrón.
Strangler Fig	Migración Core Tradicional → Core Digital	Permite trasladar productos por etapas, manteniendo un único propietario y verificando cada corte antes de ampliar el alcance
Circuit Breaker / Bulkhead	Llamadas a Riesgo, Fraude, KYC (externos)	Limitan la espera y aíslan fallos para evitar su propagación. Las verificaciones obligatorias de una operación financiera no se omiten por indisponibilidad de una dependencia.
API Composition	Consultas agregadas mediante el API Gateway	Combina respuestas de consulta para presentar información consolidada. Las escrituras y decisiones de negocio permanecen en los dominios responsables.

Ejercicio de Arquitectura de Integración

Patrón / Tecnología	Dónde se aplica	Por qué
Transactional Outbox	Core Digital y Payment Hub	Guarda el cambio de negocio y el evento pendiente en una misma transacción local. Un publicador entrega el evento al broker con reintentos; los consumidores toleran duplicados.
Anti-Corruption Layer	ACL Core Tradicional dentro de Payment Hub	Adapta el contrato del legado. La identificación y consulta de operaciones dependen de una referencia verificable que el core pueda conservar y consultar.
Saga orquestada	Payment Hub	Coordina los pasos del pago y sus compensaciones mediante transacciones locales, sin exigir una transacción global entre sistemas.

Principios reflejados: cada patrón sostiene cohesión, bajo acoplamiento, responsabilidades claras y evolución independiente. Ningún microservicio comparte base de datos con otro; el API Gateway no contiene reglas de negocio ni orquesta procesos multi-dominio; los eventos llevan payload de referencia suficiente para no forzar consultas síncronas adicionales en el camino crítico.

9.1 Tecnologías propuestas

Las tecnologías indicadas constituyen una propuesta de implementación. Su adopción debe validarse contra las capacidades del equipo, licenciamiento, soporte y objetivos de disponibilidad y latencia del banco.

Tecnología	Aplicación	Motivo de selección
Plataforma de API Management	API Gateway y gestión de APIs	Reúne políticas de acceso, control de consumo, publicación y seguimiento del ciclo de vida de las APIs.
Java con Spring	Servicios del Core Digital	Permite estructurar APIs, validaciones, acceso transaccional a datos y adaptadores de integración. Su selección debe corresponder al modelo de implementación del core y al soporte disponible.
Oracle	Persistencia transaccional del Core Digital y Payment Hub	Soporta las transacciones locales necesarias para mantener consistencia entre datos de negocio, registros de idempotencia y outbox. Cada servicio conserva almacenamiento propio y acceso exclusivo a sus datos. La elección debe justificar licenciamiento y operación.
Apache Kafka	Event Bus	Permite conservar y distribuir eventos, desacoplar consumidores y reprocesar información. El particionado, la retención y la replicación se dimensionan según volumen y recuperación; el orden se gestiona dentro de cada partición.
OpenTelemetry y plataforma de observabilidad	Métricas, logs y trazas	Proporcionan instrumentación y correlación entre componentes. Si la plataforma también conserva el historial de interacción, este se trata como un repositorio lógico con políticas propias de acceso, retención y recuperación.

Tecnología	Aplicación	Motivo de selección
Node.js	BFF de los canales	Permite coordinar llamadas de entrada y salida y adaptar respuestas para los canales web y móvil.
Kotlin sobre JVM	Payment Hub	Permite implementar la coordinación de pagos con bibliotecas del ecosistema JVM y convenciones comunes de desarrollo y operación.
Almacén clave-valor replicado, por ejemplo Redis	Registro de Propiedad de Cuentas	Permite consultar las asignaciones por identificador. La elección debe cumplir los objetivos de recuperación; la protección de las escrituras se completa con la validación local en cada core.

GraphQL se utiliza en la interfaz entre los canales y el BFF, documentada mediante su propio esquema GraphQL. Las APIs de servicios y Open Finance utilizan REST sobre HTTPS y contratos JSON, documentados con OpenAPI.

10. Estrategia de Integración Multicore

Para realizar la integración del Core Bancario tradicional y el nuevo core digital, se realizará un enrutamiento por regla de propiedad, considerando que cada producto pertenece a un solo core, que responde por sus datos y procesa sus operaciones. .

Los productos creados después de la fecha de corte del programa de migración se originan en el Core Digital. Esta regla también aplica a las iniciativas digital-first que se habiliten. Los productos existentes permanecen en el Core Tradicional hasta completar su migración. Consultar o utilizar un producto desde la banca digital no cambia su core propietario. .

Registro de propiedad

El Registro de Propiedad de Cuentas identifica qué core administra cada cuenta. Para préstamos u otros productos se utiliza el identificador correspondiente. Además del propietario, guarda una versión que aumenta con cada cambio de asignación y el estado del producto: activo o en migración.

El API Gateway consulta esta información para enrutar las solicitudes. El Ledger Adapter de Payment Hub utiliza la misma API antes de ejecutar movimientos. Ambos se apoyan en una fuente común de propiedad.

El equipo de la plataforma de migración administra el registro. Las altas de productos y los cambios de propietario se realizan mediante una API administrativa con autenticación, permisos y auditoría.

Control de escritura

Antes de aceptar una escritura, cada core comprueba que esté habilitado como propietario y que la versión recibida coincida con su versión local. Si alguna condición falla, rechaza la operación sin modificar los datos. Cambiar el registro central no deshabilita automáticamente al core anterior.

Cambio de propietario

Durante la migración, solo un core puede estar habilitado para aceptar nuevas escrituras sobre el producto. El corte se realiza en este orden:

1. Se impide iniciar nuevas operaciones y se completan o compensan las que estaban en curso. En esta etapa solo se permiten las acciones necesarias para cerrarlas.
2. Se confirma el bloqueo total de escritura en el origen. Después se transfiere y verifica la información en el destino, que permanece bloqueado durante la preparación.
3. Se registra el nuevo propietario con una versión superior y se habilita el core de destino.

4. Se comprueba que la asignación central y los controles locales de ambos cores coincidan. Solo entonces el producto vuelve al estado activo y se reabren las operaciones.

Recuperación de un corte interrumpido

La herramienta de migración guarda un registro persistente con el identificador del proceso, el producto, los cores de origen y destino, las versiones y las acciones confirmadas. Cada sistema se actualiza por separado, por lo que un fallo puede dejar parte del cambio aplicada.

Si el corte se interrumpe, las operaciones permanecen suspendidas mientras se revisa qué ocurrió en cada sistema. El equipo utiliza el registro de migración y las versiones y habilitaciones locales para completar o revertir los pasos pendientes. El indicador de escritura de un core no basta para decidir quién es el propietario.

Disponibilidad y recuperación del registro

Si no puede obtenerse una asignación válida, se bloquean las operaciones que necesitan identificar al core propietario. No se elige un core por defecto. Se realizan reintentos limitados y, si el problema continúa, se gestiona como incidente.

El registro cuenta con la estrategia de HA/DR descrita en la sección 15. Tras una restauración, las asignaciones se contrastan con las versiones y habilitaciones locales de los cores y con el registro de migración antes de reanudar el enrutamiento.

Consultas y agregación multicore

Para presentar información consolidada, la plataforma combina respuestas de APIs de consulta o utiliza la Vista Agregada de Cliente, alimentada por eventos y captura de cambios de ambos cores. Las escrituras se mantienen en el core propietario.

El API Gateway se limita a combinar las respuestas de consulta. Las decisiones y los procesos de negocio siguen en los dominios responsables. Como la Vista Agregada puede tener un desfase respecto de los cores, no se utiliza para validar fondos ni autorizar movimientos.

11. Gestión de Identidad y Acceso

- **Autenticación:** se utilizará OAuth2/OIDC como estándar en todos los canales; MFA obligatorio para operaciones sensibles (pagos de alto monto, cambio de datos de contacto).
- **Autorización:** modelo basado en roles/scopes (RBAC) para operaciones internas; modelo de consentimiento granular por scopes para terceros Open Finance, alineado a estándares FAPI (Financial-grade API).
- **Gestión de tokens:** tokens de acceso de corta duración (JWT, 5–15 min) con refresh token; revocación centralizada en la Plataforma IAM.
- **Identidad de servicio:** mTLS entre containers internos (Core Digital, Payment Hub, Event Bus); cada dominio tiene su propia service account, sin credenciales compartidas.
- **Consentimiento Open Finance:** capturado, versionado y con expiración explícita en la Plataforma IAM; el API Gateway valida el scope del consentimiento en cada llamada delegada de un tercero autorizado.

11.1 Continuidad ante degradación del IAM

Cachear las claves públicas de verificación (JWKS) en el API Gateway solo demuestra que un token fue emitido y no ha sido alterado; no demuestra que la sesión o el consentimiento sigan vigentes en este momento (un token con firma válida puede corresponder a una sesión ya cerrada o a un consentimiento ya revocado). Por eso se distingue explícitamente:

- **Operaciones de baja sensibilidad (consultas sobre datos no sensibles):** pueden aceptar la verificación de firma vía JWKS cacheado durante una degradación breve del IAM. La política depende de la sensibilidad del dato y del permiso involucrado, no de si la operación es "de solo lectura": una lectura de datos sensibles ya expuesta no es reversible aunque la operación en sí no modifique nada. Por eso este nivel se reserva explícitamente para consultas de bajo impacto.

- **Operaciones de alta sensibilidad (pagos, consultas que exponen datos personales o financieros sensibles, y cualquier llamada que dependa de un consentimiento Open Finance):** exigen una verificación de vigencia en tiempo real contra la Plataforma IAM (estado de sesión y de consentimiento), no solo la firma. Si el IAM no responde, el sistema falla cerrado (fail-closed): estas operaciones se rechazan y no se procesan en modo degradado, debido a su impacto ante una revocación no detectada.
- **Mitigación del tiempo de bloqueo:** una lista de revocación local, de vigencia corta y refrescada periódicamente desde el IAM, puede reducir la ventana de fail-closed, pero se trata siempre como información best-effort, no autoritativa. Nunca sustituye la verificación en tiempo real para pagos.

12. Seguridad, Cumplimiento Normativo y Protección de Datos

El ejercicio se resuelve para Ecuador, bajo la Ley Orgánica de Protección de Datos Personales (LOPD). Esta sección identifica las obligaciones exigidas por la LOPDP y dónde se materializa cada una en la arquitectura. No sustituye una validación legal formal: se recomienda revisión por el equipo de cumplimiento/legal del banco antes de certificar conformidad.

12.1 Obligaciones LOPDP y su mecanismo en el diseño

Obligación LOPDP	Mecanismo en la arquitectura
Base de legitimación del tratamiento (consentimiento, ejecución contractual u obligación legal, según el caso)	Cada llamada al API Gateway lleva asociada la base de legitimación del dato tratado; el consentimiento explícito (incl. Open Finance) se captura y versiona en la Plataforma IAM, única fuente autoritativa de consentimiento en toda la plataforma.
Registro de Actividades de Tratamiento (RAT)	Cada dominio (Originación, Pagos, Riesgo, Fraude, KYC) documenta qué datos personales trata, con qué finalidad y por cuánto tiempo, como parte de su contrato de dominio publicado en el catálogo.
Evaluación de Impacto (EIPD) para tratamientos de alto riesgo	El scoring de riesgo crediticio y el scoring de fraude califican como tratamiento de alto riesgo (decisión con efecto significativo sobre la persona) y requieren EIPD antes de operar en producción.
Decisiones automatizadas con efecto significativo	Riesgo y Fraude no pueden rechazar/bloquear de forma puramente automática sin vía de revisión: toda decisión adversa (rechazo de crédito, bloqueo de pago por fraude) genera un caso con posibilidad de revisión humana, no solo un score.
Notificación de incidentes de seguridad	El Event Bus y el logging centralizado permiten reconstruir el alcance de un incidente (qué datos, qué sistemas, qué período) como insumo para la notificación a la autoridad de control y a los titulares en el plazo que exige la LOPDP; el plazo exacto debe confirmarse con el equipo legal vigente.
Transferencias internacionales de datos	Motor de Riesgo y Motor de Fraude son vendors externos: si procesan datos fuera de Ecuador, requieren cláusulas contractuales de transferencia (o mecanismo equivalente reconocido) antes de integrarse. Este requisito se valida en el proceso de alta de proveedor, no solo en el diseño técnico.
Derechos del titular (acceso, rectificación, eliminación, oposición, portabilidad)	El servicio de Gestión de Derechos del Titular se implementa en la Plataforma KYC. Recibe las solicitudes mediante el API Gateway y coordina su atención con los sistemas responsables de los datos. Los consentimientos permanecen bajo la responsabilidad de IAM.
Minimización y limitación de finalidad	El BFF y el API Gateway exponen contratos de API con "vistas" específicas por consumidor, nunca el modelo completo del core.

Responsable y SLA de atención de derechos del titular: el equipo de Cumplimiento es propietario del servicio de Gestión de Derechos del Titular y del SLA de respuesta al titular; cada solicitud genera un caso trazable (mismo mecanismo de correlación que el resto de la plataforma, sección 17) desde su ingreso hasta su cierre.

Responsable y procedimiento ante incidentes de seguridad: el equipo de Seguridad/SRE es propietario de la detección (vía alertas de la Plataforma de Observabilidad, sección 17) y contención inicial; ante indicio de exposición de datos personales, escala de inmediato a Cumplimiento/DPO, quien determina si corresponde notificar a la Superintendencia de

Ejercicio de Arquitectura de Integración

Protección de Datos Personales y a los titulares, y en qué plazo. La reconstrucción del alcance del incidente se apoya en el logging estructurado con correlation ID (sección 17).

12.2 Controles técnicos de soporte

- **Cifrado en tránsito:** TLS 1.2+ en todos los puntos de integración (Cliente–BFF, BFF–Gateway, Gateway–dominios, dominios–externos).
- **Cifrado en reposo:** AES-256 en cada sistema de registro; cifrado de payloads sensibles en el Event Bus.
- **Pseudonimización:** datos identificativos tokenizados en el Event Bus y en la Vista Agregada de Cliente; la re-identificación solo es posible en el sistema de registro autoritativo.
- **Auditoría:** todo acceso a datos personales queda registrado (quién, qué campo, cuándo, finalidad), no solo en el API Gateway, sino también en las llamadas dominio-a-dominio (p. ej., Payment Hub → Core, Core → Motor de Riesgo) y en los consumidores de eventos del Event Bus, que acceden a datos personales sin pasar por el Gateway. Cada dominio y cada consumidor de eventos instrumenta el mismo estándar de logging de acceso, correlacionado por el mismo mecanismo de correlation ID (sección 17).
- **Retención:** políticas de retención definidas por dominio (p. ej., transacciones de pago retenidas según normativa financiera, casos de fraude retenidos según política de riesgo), con purgado automático en cada sistema de registro y también en las proyecciones y copias derivadas (Vista Agregada de Cliente, logs estructurados con datos personales, Historial de Interacción en la Plataforma de Observabilidad), que no quedan exentas solo por ser de solo lectura o por vivir en una plataforma primariamente técnica.

13. Estrategia de APIs y Mensajería

13.1 APIs

- **APIs síncronas (REST/HTTPS + JSON):** para operaciones que requieren respuesta inmediata (saldo, autenticación, screening de fraude).
- **APIs externas (Open Finance):** siguiendo estándar FAPI / especificaciones Open Finance, versionadas (/v1/, /v2/) con período de convivencia de 6–12 meses entre versiones antes de deprecarse.

Ejemplo de contrato — POST /v1/payments (Payment Hub)

```
{
  "idempotencyKey": "b3f1e2a0-3c1d-4e9a-8b21-...",
  "debtorAccountId": "ACC-DIG-00019284",
  "creditorAccountId": "ACC-LEG-00445190",
  "amount": { "value": 250.00, "currency": "USD" },
  "paymentType": "INTERNAL_TRANSFER",
  "initiatedBy": { "channel": "MOBILE_APP" }
}
```

La clave de idempotencia es obligatoria y la genera el cliente (app/BFF); el Payment Hub la rechaza si falta.

Otros endpoints (resumen)

- GET /v1/accounts/{id}/balance (Core Digital, vía Gateway): consulta saldo
- POST /v1/risk/credit-assessment (Motor de Riesgo): solicita evaluación de riesgo
- POST /v1/fraud/screen (Motor de Fraude): solicita score de fraude en tiempo real
- POST /openfinance/v1/consents (IAM): crea consentimiento AISP/PISP

13.2 Mensajería / Eventos y el patrón Outbox

El riesgo central de una arquitectura orientada a eventos en pagos es el problema de "dual write": si el servicio escribe su estado en la base de datos y luego publica el evento como dos operaciones separadas, una puede tener éxito y la otra fallar (evento perdido o evento huérfano sin estado real detrás).

Solución mediante Transactional Outbox: el estado del pago y el evento a publicar se escriben en la misma transacción de base de datos, en una tabla "outbox" dentro del propio Payment Store. Un proceso Outbox Publisher (o Debezium/CDC sobre esa tabla) lee la tabla de forma asíncrona y publica al Event Bus con reintentos hasta confirmar, marcando la fila

Ejercicio de Arquitectura de Integración

como publicada. Si el proceso falla a mitad de camino, el evento sigue pendiente en la tabla y puede reintentarse. La entrega puede producir duplicados, por lo que los consumidores deben aplicar deduplicación (ver diagrama de componentes, sección 8.3).

Condiciones para la deduplicación del lado del consumidor: la fórmula "at-least-once + deduplicación = exactly-once" solo es válida si se cumplen tres condiciones; enunciarla sin ellas es insuficiente:

- **1. Registro de deduplicación + efecto local en una misma transacción, cuando sea posible:** si el efecto del evento es una escritura local (p. ej., actualizar la Vista Agregada), el consumidor inserta el eventId en su propia tabla de "eventos procesados" y aplica el efecto en la misma transacción de base de datos. Si cualquiera de las dos falla, ambas se revierten. Así se evita que quede un efecto aplicado sin su registro de deduplicación, o viceversa.
- **2. Para llamadas remotas (el efecto no es una escritura local sino invocar otro sistema, p. ej. Payment Hub llamando a debitar en un core):** no existe una transacción compartida entre Payment Hub y el core, por lo que el registro local de "ya procesado" no es suficiente por sí solo. Se requiere una clave de operación estable (operationId) enviada en cada llamada, y el sistema remoto debe soportar consulta de estado por esa clave (ver el procedimiento de resultado incierto en la sección 13.3).
- **3. Solicitudes concurrentes con la misma clave:** si dos solicitudes llegan con el mismo idempotencyKey mientras la primera sigue en curso, la segunda no se procesa en paralelo. El Idempotency Guard la rechaza (o la hace esperar) según el estado registrado (PENDING/COMPLETED/FAILED) para esa clave, evitando una carrera que dos ejecuciones concurrentes sí producirían.

Con estas tres condiciones, Transactional Outbox sigue siendo la base del mecanismo. Su alcance debe precisarse: outbox ofrece entrega eventual con reintentos, condicionada a la conservación del dato en la tabla outbox y a la recuperación del publicador o del broker. No es una garantía absoluta de "nunca cero veces" (una pérdida irrecoverable de la fila outbox, o un publicador que nunca se recupera, sí puede perder el evento). Permite además duplicados en el broker (reintentos de red, rebalanceo del consumidor) y por eso exige consumidores idempotentes según las condiciones anteriores, no los sustituye.

- **Esquema versionado:** registro de esquemas (schema registry) con JSON Schema o Avro.
- **Reintentos y DLQ:** backoff exponencial en el consumidor; tras N intentos fallidos el evento va a una Dead Letter Queue para revisión manual, sin bloquear el resto del tópico.
- **Ordenamiento:** partición por ID de cuenta/cliente cuando el orden de eventos de una misma entidad importa.
- **Política ante DLQ: pausa de la partición completa.** Enviar un evento a DLQ y continuar con el resto del tópico no es suficiente por sí solo, porque permitiría que un evento posterior de la misma cuenta adelante al que falló, rompiendo el orden de esa entidad. Kafka no ofrece de forma nativa "saltar una clave y seguir con las demás" dentro de una partición sin un mecanismo adicional de almacenamiento diferido; en vez de construir esa capa extra, se opta por pausar la partición completa donde ocurrió el fallo hasta resolver el evento en DLQ. Esto es una decisión consciente, no una limitación oculta: como el particionado es por cuenta (sección 13.2), pausar una partición bloquea también las demás cuentas asignadas a esa misma partición, no solo la cuenta afectada. El tamaño de ese radio de impacto depende del número de particiones configuradas, y es un parámetro a dimensionar según el volumen esperado, no algo que este documento fije de antemano.

13.3 Resultado incierto de una llamada remota (timeout en un débito)

Caso concreto: Payment Hub llama a Ledger Adapter para debitar una cuenta; el core la procesa correctamente, pero la respuesta se pierde por una falla de red antes de llegar a Payment Hub. Si Payment Hub reintenta con una llamada de débito ordinaria, el riesgo es debitar dos veces. El outbox de Payment Hub no cubre este caso: outbox resuelve la publicación de eventos propios, no la incertidumbre de una llamada síncrona saliente hacia otro sistema.

- Toda operación de movimiento de fondos lleva un operationId estable, generado por Payment Hub y conservado en todos sus reintentos. Cada paso de una orden (reserva, débito, crédito o compensación) tiene un identificador de operación distinto, vinculado al mismo paymentId. Antes de ejecutar efectos externos, Payment Hub persiste la relación entre el cliente o aplicación solicitante, el idempotencyKey, el paymentId, los datos de la orden y los operationId correspondientes. La misma clave de idempotencia dentro del mismo ámbito identifica una única orden; si se reutiliza con datos diferentes, la solicitud se rechaza.
- **Ante timeout, Payment Hub NO reintenta a ciegas:** primero consulta el estado de esa operación en el core (GET por operationId).

Ejercicio de Arquitectura de Integración

- **Si el core confirma que se aplicó:** Payment Hub la trata como exitosa y continúa el esquema sin reintentar el débito.
- **Si el core confirma que no se aplicó (o no la encuentra):** es seguro reintentar con el mismo operationId, porque el core debe tratarlo como idempotente.
- **Si el estado sigue sin poder determinarse tras reintentar la consulta:** el esquema no avanza. El caso pasa a verificación manual (cola de reconciliación), y los fondos permanecen en estado de retención hasta resolverse; nunca se asume éxito ni se libera la retención por defecto.

Requisito arquitectónico derivado: tanto el Core Digital como el Core Tradicional deben exponer, para las operaciones de movimiento de fondos, (a) un parámetro de operationId idempotente y (b) un endpoint de consulta de estado por esa clave. El Core Digital puede diseñarse así desde el inicio; el Core Tradicional, al ser legado, puede no soportarlo de forma nativa. En ese caso se requiere una capa anticorrupción (patrón Anti-Corruption Layer, sección 9) que añada ambas capacidades delante de su API antes de integrarlo al esquema de pagos. Sin esa capa, el Core Tradicional no debería recibir llamadas de débito/crédito desde Payment Hub sin control adicional de reconciliación manual.

Esa capa se modela como un componente propio dentro de Payment Hub, denominado "ACL Core Tradicional" (ver diagrama de componentes, 8.3). Se mantiene separado de Ledger Adapter: Ledger Adapter mantiene una lógica genérica de débito/crédito válida para cualquier core, mientras el ACL concentra exclusivamente las particularidades del legado (traducción de protocolo, adición de operationId, consulta de estado). Ledger Adapter llama al Core Digital directamente y al Core Tradicional únicamente a través del ACL.

Precondición que el ACL no puede suplir por sí solo: agregar un adaptador no garantiza automáticamente idempotencia. El ACL solo puede ofrecer operationId y consulta de estado si el Core Tradicional expone, de alguna forma, una manera confiable de identificar la operación ya ejecutada (p. ej., una referencia externa que el legado persista y permita consultar, o un mecanismo equivalente). Si el legado no ofrece nada de eso, el ACL no puede inventarlo: en ese caso el resultado de un timeout se mantiene incierto por diseño y va siempre a conciliación manual, en vez de presentarse como un caso resuelto por la sola existencia del adaptador. Confirmar esta capacidad del Core Tradicional específico es una validación pendiente antes de comprometerse con este diseño.

Ejemplo de evento — payment.confirmed.v1

```
{
  "eventId": "9a6e7c3b-...",
  "eventType": "payment.confirmed.v1",
  "correlationId": "req-7f2a...",
  "occurredAt": "2026-09-06T14:32:10Z",
  "paymentId": "PAY-00058231",
  "debtorAccountId": "ACC-DIG-00019284",
  "creditorAccountId": "ACC-LEG-00445190",
  "amount": { "value": 250.00, "currency": "USD" },
  "status": "CONFIRMED"
}
```

Otros eventos (resumen)

- account.opened.v1
- payment.initiated.v1 / payment.failed.v1
- fraud.case.created.v1
- legacy.party_reference.updated.v1 (CDC desde el core tradicional; referencia/vínculo cuenta-parte, no perfil autoritativo: IAM sigue siendo la única fuente de perfil, sección 7)

14. Modelo de Gobierno de APIs y Microservicios

- **Catálogo centralizado:** el API Gateway/API Manager es la fuente única de verdad, con portal de desarrolladores para las APIs Open Finance.
- **Ciclo de vida API-first:** diseño (contrato OpenAPI/AsyncAPI revisado antes de construir) → build → test → publish → deprecate, con revisión de un comité de arquitectura/seguridad/producto.
- **Estándares obligatorios:** nomenclatura /v{n}/{recurso}, manejo uniforme de errores (RFC 9457, que reemplaza a RFC 7807), paginación estándar, versionado semántico.

Ejercicio de Arquitectura de Integración

- **Ownership:** cada dominio/microservicio tiene un equipo propietario responsable de su API y sus SLAs ("you build it, you run it").
- **Reglas de microservicios:** ningún servicio accede a la base de datos de otro; los cambios de contrato son backward-compatible o pasan por período de convivencia; todo contrato se publica en el catálogo antes de integrarse.
- **Límite Gateway vs. orquestación de negocio:** el API Gateway enruta y aplica políticas técnicas (authN/Z, rate-limiting, versionado de contrato); coordinar los pasos de un proceso de negocio (p. ej., el esquema de pago) es responsabilidad del dominio propietario del proceso (Payment Hub), nunca del Gateway. Esta separación se revisa en cada diseño de API que pase por el comité de gobierno.
- **Métricas de gobierno:** adherencia a SLA, % de APIs con contrato publicado, tiempo de vida promedio de una versión deprecada.

15. Alta Disponibilidad y Recuperación ante Desastres

La estrategia se define por nivel de criticidad de cada container y dependencia externa:

Componente	Criticidad	Estrategia HA/DR
BFF / API Gateway	Alta (punto único de entrada)	Sin estado de servidor: la sesión vive en el JWT emitido por IAM, no en memoria del BFF; multi-AZ, autoscaling, sin estado de servidor que recuperar.
Core Bancario Digital	Alta	Despliegue multi-AZ con réplica síncrona para proteger los movimientos confirmados ante fallos de zona. Recuperación en una región secundaria mediante réplica asíncrona. RTO objetivo: 15 minutos. Ante pérdida de región, el RPO depende del retraso de replicación y debe medirse y validarse mediante simulacros. Antes de reanudar operaciones, se concilian los movimientos del período afectado. Esta estrategia protege la persistencia contable del Core Digital y es independiente de la recuperación de órdenes de Payment Hub.
Payment Hub	Alta	Despliegue multi-AZ y recuperación en una región secundaria. RTO objetivo: 15 minutos. RPO objetivo: cero para el registro de órdenes y operaciones, con persistencia confirmada en la región de recuperación antes de ejecutar movimientos; hasta un minuto para el estado de ejecución reconstruible. Tras un desastre, conciliar con los cores y rieles antes de reanudar los pagos afectados. Objetivos sujetos a validación mediante simulacros.
Registro de Propiedad de Cuentas	Alta (su indisponibilidad bloquea las operaciones que requieren identificar al core propietario)	Réplica multi-AZ. RTO objetivo: 5 minutos; RPO objetivo: cero. Si no existe una asignación válida, las operaciones dependientes fallan cerrado. Antes de reabrir el enrutamiento, el equipo de plataforma de migración verifica las asignaciones restauradas contra las versiones y habilitaciones locales de ambos cores y el registro durable de migración.
Core Bancario Tradicional (externo)	Alta (autoritativo para productos no migrados)	Su HA/DR la gestiona el equipo propietario del legado; se exige un SLA de disponibilidad y un RTO/RPO documentado como precondition de integración, no un objetivo que la nueva plataforma pueda garantizar por sí sola.
Plataforma IAM (externo)	Alta (sin autenticación no hay acceso a ningún flujo)	Se exige alta disponibilidad multi-región al proveedor; el API Gateway cachea localmente las claves públicas de verificación de JWT (JWKS) con TTL corto, para tolerar una degradación breve del IAM sin bloquear la validación de tokens ya emitidos.

Componente	Criticidad	Estrategia HA/DR
Event Bus	Alta (backbone)	Clúster replicado (factor ≥ 3), particionado por dominio, multi-AZ, mirroring a región DR
Vista Agregada de Cliente	Media	RPO más flexible, dado que la vista puede reconstruirse desde el Event Bus si se pierde
Motor de Riesgo, Motor de Fraude, KYC (externos)	Depende de SLA del proveedor	Circuit breaker + degradación controlada (nunca fallo abierto sin control en pagos)

Los RTO y RPO son objetivos de diseño que se validarán mediante simulacros periódicos. Se medirán el tiempo de recuperación, la pérdida de datos y la coherencia del estado restaurado. Las operaciones afectadas se reanudarán después de completar las verificaciones y conciliaciones correspondientes.

Conciliación tras un failover de Payment Hub: para recuperar una orden no basta con comprobar que hubo un movimiento contable. Se necesita conservar su identidad, los datos originales y la relación con cada operación ejecutada. Payment Hub mantiene esta información en un registro durable de órdenes y operaciones dentro de su almacenamiento propio. La recuperación de este registro es una precondition para reanudar los pagos afectados.

La identidad y los datos de una orden aceptada deben quedar protegidos frente a la pérdida de la región principal antes de ejecutar movimientos externos. Para ese registro se establece un objetivo de RPO cero, cuya implementación requiere persistencia confirmada en el destino de recuperación y validación mediante pruebas de desastre. El RPO referencial de un minuto aplica al estado de ejecución reconstruible; no a la información necesaria para identificar las órdenes aceptadas. Esta distinción debe reflejarse en la configuración de persistencia y replicación de Payment Hub.

Cada core conserva el operationId como referencia del movimiento y permite consultar su resultado, sujeto a la precondition del legado descrita en 13.3. Tras un failover, el proceso de conciliación recupera las órdenes y sus operaciones, consulta los resultados en los cores y reconstruye el estado de ejecución. Para pagos externos, también verifica el resultado con el riel correspondiente mediante su referencia de operación. Un resultado incierto permanece pendiente de conciliación; no se interpreta como una operación no ejecutada ni se reintenta a ciegas.

Durante este proceso se bloquea el reprocesamiento de las órdenes afectadas y se mantienen las restricciones necesarias sobre sus cuentas hasta resolver las operaciones pendientes. Una transformación determinística puede utilizarse para generar identificadores reproducibles, pero no se presupone reversible ni sustituye el registro durable de las relaciones entre identificadores.

16. Plan de Migración Gradual (Strangler Fig)

- **Fase 0. Preparación:** desplegar el API Gateway/BFF delante del Core Tradicional sin migrar nada; todo el tráfico sigue al legado, pero ya bajo control de enrutamiento centralizado.
- **Fase 1. Digital-first para lo nuevo:** todo producto nuevo se origina directo en el Core Digital, según la regla de corte.
- **Fase 2. Migración de bajo riesgo:** mover un segmento acotado (por ejemplo: cuentas de ahorro universales o básicas sin otros productos vinculados) del core tradicional al core digital, validando el patrón end-to-end (CDC, eventos, read model) a pequeña escala.
- **Fase 3. Migración progresiva por dominio:** repetir para cuentas corrientes y luego préstamos, priorizando por menor complejidad/interdependencia.
- **Fase 4. Retiro del core tradicional:** cuando se hayan migrado todos los productos activos en el core tradicional, se deberá evaluar el retiro completo o mantenerlo como repositorio de solo lectura para cumplimiento histórico.

16.1 Migración de una cuenta (Fases 2–3): procedimiento con corte controlado

Mover una cuenta del legado al core digital no es una operación instantánea ni reversible por switch de ruteo. Requiere una ventana de corte controlada:

Ejercicio de Arquitectura de Integración

- Congelamiento de escritura: la herramienta de migración impide que el Core Tradicional inicie nuevas operaciones sobre esa cuenta (ventana corta, comunicada). No basta con dejar de enrutarle tráfico; el propio core deja de aceptar operaciones nuevas aunque alguien se las envíe directamente. Durante esta etapa, solo se permiten las acciones controladas necesarias para completar o compensar las operaciones que ya estaban en curso (p. ej., un pago en estado pendiente), sin dejarlas a medio proceso. Una vez resueltas, se confirma el bloqueo completo de escritura del Core Tradicional para esa cuenta, antes de continuar al siguiente paso.
- Snapshot y transferencia: se copia el estado (saldo, historial, productos vinculados) del core tradicional al core digital.
- Verificación de paridad: un job compara saldo y últimos movimientos en ambos cores antes de continuar.
- Corte de propiedad: la herramienta de migración registra el inicio de esta acción en su registro durable de migración (identificador de migración, producto, origen, destino, versión) y ejecuta por separado la actualización del Registro de Propiedad de Cuentas y la activación local de "habilitado para escritura" en el Core Digital para la nueva versión. Cada una de las dos acciones se confirma y se anota individualmente en el registro durable de migración a medida que se completa.
- Reapertura de escritura: la cuenta vuelve a aceptar transacciones, ahora exclusivamente en el Core Digital, que las acepta porque su condición local ya quedó habilitada en el paso anterior.
- Modo sombra (shadow mode) posterior: durante un período de observación, las transacciones que habría procesado el legado se simulan (sin efecto real) y se comparan contra el resultado real del core digital, para detectar discrepancias antes de dar la migración por definitiva.

Recuperación de un corte interrumpido: si el proceso se detiene durante o después del paso 4 (p. ej., por una caída de la herramienta de migración), la recuperación no presupone un estado binario predefinido. Como el registro central y la habilitación local del Core Digital son acciones separadas, pudo completarse una sin la otra. La herramienta de migración usa su registro durable (qué acciones quedaron confirmadas) junto con el estado real de las condiciones locales de ambos cores y del registro central para determinar exactamente qué falta: si el registro central se actualizó pero el Core Digital no llegó a habilitarse, completa esa habilitación; si ocurrió al revés, corrige la discrepancia. La escritura (paso 5) solo se reabre cuando el registro central, la habilitación local del Core Digital y el registro durable de migración son coherentes entre sí. La reapertura requiere esta verificación y no se basa en un estado seguro supuesto de antemano.

16.2 Rollback: qué significa realmente volver atrás

Un rollback vía feature flag en el API Gateway ("volver a enrutar al legado") NO revierte el estado: si el core digital ya procesó movimientos sobre la cuenta migrada, el legado no tiene esos movimientos ni el saldo actualizado. Redirigir el tráfico sin más dejaría al cliente con dos historiales divergentes.

Por eso el rollback real tiene dos escenarios distintos:

- **Rollback antes de confirmar el paso 4 (corte de propiedad) de la sección 16.1:** es seguro y de bajo costo en la mayoría de los casos, pero no debe asumirse que el registro de propiedad permanece intacto solo porque el paso 4 no se dio por completado. Al ser dos acciones separadas (actualización del registro central y habilitación local del Core Digital), es posible que una haya quedado aplicada parcialmente antes de interrumpirse. Este rollback verifica el registro durable de migración y el estado real de ambos cores, y revierte cualquier actualización parcial encontrada, antes de dar la migración por abortada. No debe asumirse que nada cambió sin realizar estas verificaciones.
- **Rollback después del corte de propiedad (la cuenta ya opera en el core digital y tiene movimientos propios):** requiere una migración inversa con el mismo procedimiento de la sección 16.1 pero en sentido contrario, que incluye:
 - Congelar escritura en el core digital para esa cuenta.
 - Reconciliar: calcular la diferencia de movimientos ocurridos en el core digital desde el corte, y aplicarlos como ajustes contables trazables en el core tradicional (no un simple copy-paste de saldo final).
 - Tratar operaciones en curso (pagos en estado pendiente/en curso dentro del esquema) como casos explícitos: se completan en el core digital antes de revertir, o se compensan formalmente (nunca se abandonan a medio proceso).
 - Actualizar el Registro de Propiedad de Cuentas de vuelta al core tradicional, de forma atómica, solo después de reconciliar.

Ejercicio de Arquitectura de Integración

- Notificar al cliente si hubo algún ajuste visible en su historial.

Condición para permitir el rollback post-corte: solo se autoriza dentro de una ventana de contingencia definida (p. ej., 24–72 horas desde el corte) y mientras el volumen de movimientos posteriores sea reconciliable de forma automática; superado ese umbral, la corrección se maneja como incidente operativo con intervención manual, no como rollback de plataforma.

Mitigación adicional en cada fase: shadow mode antes de cortar tráfico real, comparación de resultados entre legado y digital durante el período de observación, y feature flag en el API Gateway que solo es "instantáneo" en el sentido correcto descrito arriba. En ninguna fase ambos cores son propietarios simultáneos del mismo producto.

17. Observabilidad y Trazabilidad

La Plataforma de Observabilidad centraliza las métricas, los logs y las trazas de la solución y se representa como sistema externo en los diagramas de contexto y contenedores. Todos los contenedores envían telemetría mediante OTLP. Para facilitar la lectura, el diagrama de contenedores muestra únicamente la conexión del API Gateway como relación representativa de este envío; la instrumentación también aplica al BFF, Core Digital, Payment Hub, Event Bus y demás contenedores.

La conexión desde el Event Bus representa la entrega de eventos de interacción publicados por el BFF, diferenciada del envío de telemetría técnica. La Plataforma de Observabilidad consume estos eventos y conserva el Historial de Interacción como repositorio lógico bajo la responsabilidad funcional del dominio de Gestión de interacciones y canales digitales. Su alojamiento en esta plataforma no modifica la propiedad del dato ni lo excluye de los controles de acceso, retención, protección de datos y recuperación establecidos para dicho historial.

- **Distributed tracing:** correlation ID de extremo a extremo (Cliente → BFF → Gateway → dominio → externos), propagado también en los eventos, permitiendo reconstruir el camino completo de una transacción.
- **Métricas:** cada container expone latencia, tasa de error y saturación hacia la Plataforma de Observabilidad.
- **Logging estructurado:** formato JSON con el mismo correlation ID, sin PII en texto plano (alineado a minimización de datos).
- **Trazabilidad de negocio:** cada evento de dominio (pago confirmado, fraude detectado) queda auditado con su ID de correlación, sirviendo tanto a observabilidad técnica como a auditoría de cumplimiento.
- **Alertas basadas en SLO:** por ejemplo, para Payment Hub se propone p99 de latencia < 2 s y tasa de error < 0.1%. Este SLO mide la aceptación, validación e inicio de ejecución dentro de Payment Hub (hasta que la orden se entrega al riel de pago). No incluye la liquidación externa, que depende del riel/red de pago correspondiente y tiene su propio SLA, típicamente mayor (p. ej., liquidación ACH puede tardar hasta el día hábil siguiente).

17.1 Responsables y procedimiento ante incidentes

- **Detección:** el equipo de Seguridad/SRE monitorea las alertas de la Plataforma de Observabilidad; un SLO incumplido o un patrón anómalo (p. ej., pico de fallos de autenticación, screening de fraude con latencia degradada) dispara la alerta inicial.
- **Clasificación y contención:** Seguridad/SRE clasifica severidad y ejecuta contención técnica inmediata (aislar el componente afectado, activar circuit breakers, si aplica).
- **Escalamiento a Cumplimiento:** si hay indicio de exposición de datos personales, se escala de inmediato al equipo de Cumplimiento/DPO, quien evalúa si corresponde notificar a la Superintendencia de Protección de Datos Personales y a los titulares, y en qué plazo, conforme a la LOPDP (sección 12).
- **Reconstrucción del alcance:** se apoya en el logging estructurado con correlation ID. Esto permite acotar qué cuentas, qué operaciones y qué ventana de tiempo estuvieron afectadas, insumo necesario tanto para la notificación regulatoria como para la comunicación a los titulares.
- **Cierre y lecciones aprendidas:** todo incidente con impacto en datos personales o en disponibilidad de pagos genera un post-mortem con acciones de seguimiento, propietario y fecha, gestionado por el mismo equipo que lo contuvo.

18. Principios de Diseño Aplicados

Las decisiones de diseño adoptadas en esta arquitectura responden principalmente a tres problemas del escenario planteado: la coexistencia de dos cores bancarios, la necesidad de evolucionar capacidades digitales sin depender del Core Bancario Tradicional y la incorporación de capacidades regulatorias y de Open Finance sin generar nuevos puntos de acoplamiento.

Por esa razón se estableció como principio fundamental que cada producto tenga un único sistema propietario. La coexistencia entre el Core Tradicional y el Core Digital no se resuelve mediante sincronización bidireccional de bases de datos, sino mediante una asignación explícita de propiedad y mecanismos de consulta agregada. Esta decisión reduce conflictos de consistencia y facilita la migración progresiva del Core Tradicional.

También se evitó concentrar lógica de negocio en los componentes de integración. El API Gateway participa exclusivamente en funciones técnicas de seguridad, control y enrutamiento. La ejecución de procesos de negocio permanece en los dominios responsables, particularmente en Payment Hub para el procesamiento de pagos. De esta manera, la evolución funcional de cada dominio puede realizarse sin depender de modificaciones en la capa de integración.

La comunicación entre dominios combina APIs síncronas y eventos asíncronos según la criticidad de cada operación. Las decisiones transaccionales requieren respuestas inmediatas y verificables, mientras que la propagación de cambios y la construcción de vistas consolidadas utilizan mecanismos orientados a eventos. Esto permite reducir dependencias temporales entre sistemas sin comprometer la integridad de las operaciones financieras.

Otro principio relevante ha sido evitar bases de datos compartidas. Cada dominio mantiene control sobre sus propios datos y publica únicamente contratos estables hacia el resto de la organización. Este enfoque facilita la autonomía de equipos, reduce dependencias entre despliegues y mejora la capacidad de evolución independiente de los componentes.

Finalmente, todas las decisiones de integración, seguridad, observabilidad y migración se evaluaron bajo el criterio de trazabilidad operativa. Cada operación debe poder reconstruirse de extremo a extremo, incluso ante fallos parciales, eventos duplicados o procesos de recuperación ante desastre. Esta necesidad explica la incorporación de mecanismos como correlation IDs, Transactional Outbox, control de idempotencia y conciliación operativa.

Las interfaces ofrecen las operaciones y los datos que necesita cada consumidor. Los adaptadores concentran las particularidades de los sistemas externos y permiten que la lógica de negocio dependa de contratos estables.

Estos contratos y adaptadores permiten probar la lógica de negocio con respuestas simuladas de sus dependencias. Así se verifican escenarios como solicitudes duplicadas, fallos de comunicación y resultados inciertos.

19. Conclusiones

El principal desafío de este escenario es mantener una convivencia entre el Core Bancario Tradicional y el Core Digital durante la transición. De esta necesidad surgen las principales decisiones de arquitectura de la propuesta.

Por esa razón, cada producto tiene un único core propietario. La integración se realiza mediante APIs, eventos y vistas agregadas, sin sincronización bidireccional entre los cores durante la operación habitual. Este enfoque exige controles de migración, observabilidad y conciliación, pero ayuda a reducir el riesgo de escrituras duplicadas y mantiene clara la responsabilidad sobre los datos.

La propuesta también separa las funciones de integración de las decisiones de negocio. El API Gateway, el Event Bus y la Plataforma de Observabilidad proporcionan servicios técnicos, mientras que las reglas y los procesos de negocio permanecen en los dominios responsables. Esta separación facilita su evolución y evita concentrar la lógica funcional en la capa de integración.

Ejercicio de Arquitectura de Integración

El diseño contempla situaciones habituales en un entorno distribuido: respuestas que se pierden después de un movimiento financiero, migraciones interrumpidas, proveedores que dejan de responder y eventos que se reciben más de una vez. Para atenderlas, incorpora idempotencia, conciliación, trazabilidad y procedimientos de recuperación desde el inicio.

Antes de implementar la propuesta, será necesario confirmar las capacidades del Core Bancario Tradicional, los acuerdos con los proveedores, los objetivos de recuperación y los requisitos regulatorios aplicables. La arquitectura establece una dirección para la modernización del banco. Llevarla a la práctica requerirá validar estos supuestos, probar los procedimientos y avanzar por etapas, utilizando los resultados de cada fase para decidir el siguiente paso.

Anexos

A. Tabla de Controles de Seguridad

Control	Capa donde aplica	Mecanismo
Autenticación fuerte (SCA/MFA)	Cliente → BFF / API Gateway	OAuth2/OIDC + MFA en operaciones sensibles
Autorización por consentimiento	API Gateway (Open Finance)	Scopes FAPI, validación de consentimiento vigente
Cifrado en tránsito	Todas las integraciones	TLS 1.2+, mTLS interno entre containers
Cifrado en reposo	Todos los sistemas de registro	AES-256
Pseudonimización	Event Bus, Vista Agregada	Tokenización de identificadores
Prevención de fraude en tiempo real	API Gateway, Payment Hub → Motor de Fraude	Screening síncrono de baja latencia
Verificación de sanciones/PEP	Payment Hub → Cumplimiento	Sanction Screening antes de ejecutar el pago
Auditoría de acceso a datos personales	API Gateway, llamadas dominio-a-dominio y consumidores de eventos (ver 12.2)	Logging estructurado con finalidad y consentimiento asociado, correlacionado por el mismo correlation ID en todos los puntos de acceso
Segregación de credenciales de servicio	Todos los containers internos	Service accounts individuales + mTLS
Retención y purgado	Cada sistema de registro, y proyecciones/copias derivadas (Vista Agregada, Historial de Interacción en Observabilidad; ver 12.2)	Políticas de retención por dominio, purgado automático; no exentas por ser de solo lectura o vivir en una plataforma primariamente técnica

B. Ejemplos de APIs (resumen)

- POST /v1/payments (Payment Hub)
- GET /v1/accounts/{id}/balance (Core Digital)
- POST /v1/risk/credit-assessment (Motor de Riesgo)
- POST /v1/fraud/screen (Motor de Fraude)
- POST /openfinance/v1/consents (Plataforma IAM)

C. Ejemplos de Eventos (resumen)

- account.opened.v1
- payment.initiated.v1
- payment.confirmed.v1
- payment.failed.v1
- fraud.case.created.v1
- legacy.party_reference.updated.v1 (CDC)

D. Entregables GitHub

El repositorio [ejercicio_integracion](#) reunirá el documento final y los diagramas que respaldan la propuesta.

1. Documento PDF: Arquitectura_Integracion_Bancaria.pdf
2. Diagramas C4 en formato draw.io : c4_bancario.drawio
3. Imagen del diagrama contexto : c4_bancario-C4 - Nivel 1 - Contexto.drawio.png

Ejercicio de Arquitectura de Integración

4. Imagen del diagrama contenedores : c4_bancario-C4 - Nivel 2 - Contenedores.drawio.png

5. imagen del diagrama componentes: c4_bancario-C4 - Nivel 3 - Componentes Payment Hub.drawio.png